

Combined Backend and App Communication Manual

This document provides a structured integration flow for onboarding and validating device communication across backend, MQTT, and BLE channels.

Step 1. Whitelisting of IP and Mobile Number

Before any integration or testing activity, complete the required whitelisting.

- **Test Server IP:** 135.235.166.209
- **Production Server IP:** 103.195.217.127
- **Authorized Mobile Number:** _____

Step 1.1 Notes

- Ensure IP whitelisting is completed on all relevant network/firewall layers.
- Use only approved mobile numbers for command/control and validation workflows.
- Maintain separate access-control records for test and production environments.

Step 2. Download and Set Up the Test Server TLS Certificate

Install and validate the TLS trust chain required for secure MQTT transport.

Step 2.1 Certificate Setup

- Download the test server CA certificate from the portal.
- Provision the certificate to device firmware/storage in the expected format.
- Enable server certificate verification in the MQTT/TLS client configuration.

Step 3. Point Devices to the Test Server IP and Mobile Number

Configure each device to use the test environment parameters.

- **Test Server IP:** 135.235.166.209
- **Test Server TLS:** {SKYTRON_TEST_MQTT_TLS_1.0.0.crt}
- **Authorized Mobile Number:** _____

Step 3.1 Required Configuration

- Set the server endpoint/IP to the **Test server IP**.
- Set the approved mobile number for command and verification use cases.

- Confirm APN/network settings allow outbound connectivity to the test endpoint.

Step 4. MQTT Integration

Step 4.1 MQTT Publish and Subscribe Session Setup

Establish secure MQTT sessions for device telemetry, emergency signaling, and command handling.

- Protocol: MQTTS over TLS
- Typical Port: 8883
- Client identity derived from IMEI (username/client ID pattern)
- QoS and reconnect policy as defined in firmware

Step 4.2 MQTT Topics for Each Data Packet

Use IMEI-scoped topics consistently.

For device IMEI `${imei}`:

Purpose	Topic	Direction
Tracking / LOGIN / OT / health stream	<code>deviceTracking/\${imei}</code>	Device → Server
Emergency EPB stream	<code>deviceEM/\${imei}</code>	Device → Server
Backend commands	<code>deviceResponse/\${imei}</code>	Server → Device
Command replies / ACK	<code>deviceResponseAck/\${imei}</code>	Device → Server

Step 4.3 MQTT Packets, Commands, and Replies

Device-to-Server Packet Categories

- Tracking client:
 1. Login packet
 2. Health packet
 3. Tracking packet
- Emergency client:
 1. Login packet
 2. Emergency packet

A) Login Packet

`${veh_reg_no},${imei},${fw_version},${proto_version},${lat}<N|S><lon><E|W>`

When sent:

- Tracking session init
- Emergency session init

B) PVT Tracking / Alert Packet

```
$,PVT,<vendor_id>,<fw_version>,<pkt_type>,<alert_id>,<L|H>,<imei>,<veh_reg_no>,<gps_fix>,<date>,<time>,<lat>,<lat_dir>,<lon>,<lon_dir>,<speed>,<heading>,<sats>,<alt>,<pdop>,<ignition>,<main_power>,<veh_input_v>,<int_batt_v>,<emergency_status>,<box_tamper>,<csq>,<mcc>,<mnc>,<lac>,<cellid>,<nbr1_cell>,<nbr1_lac>,<nbr1_sig>,...,<nbr4_cell>,<nbr4_lac>,<nbr4_sig>,<din_bitmap>,<dout_bitmap>,<frame6>,<odometer>,<checksum>,*
```

C) Health Packet

```
$,HLM,<vendor_id>,<fw_version>,<imei>,<batt_pct>,<low_batt_thresh_pct>,<mem_used_pct>,<inter>
```

D) Emergency EPB Packet

```
$,EPB,<EMR|SEM>,<imei>,<NM|SP>,<date><time>,<A|V>,<lat>,<lat_dir>,<lon>,<lon_dir>,<alt>,<spe>
```

- EMR = SOS start
- SEM = SOS stop
- NM = normal immediate, SP = stored/history mode

E) OT Command Audit Packet Base is PVT with:

- pkt_type=OT
- alert_id=12

Appended payload info:

```
<source>,<source_info>,<clean_command>,<clean_reply>
```

Where source is one of SMS, UART, SERVER.

E) SOS-specific behavior:

- SOS start: send EA,10 (tracking) + EPB,EMR (emergency).
- SOS stop: send EA,11 (tracking) + EPB,SEM (emergency).

Server-to-Device Command Model

- JSON command payloads are received from backend command topic. Supported JSON fields:
- {"keys": "@SETSOSDIS-1*"} → trigger SOS stop/disable
- {"keys": "@GETLOC*"} → run command parser

Device Reply Model

- Command execution results are published to the ACK topic.
- Reply payload examples include success, error, and status/progress messages.

For backend-originated commands (`source=SERVER`), reply is published on:

- `deviceResponseAck/${imei}`

Typical reply strings:

- `SET SUCCESSFULLY ...`
- `REGNO-..., GETLOC-..., etc.`
- `ERR:<reason>`
- `FOTA STARTED, FOTA BUSY`

Step 5. BLE Communication

Step 5.1 BLE Setup

Configure BLE as a peripheral channel for app-device interaction.

- Initialize BLE service/characteristics per firmware design.
- Enable app-side notifications before issuing write commands.
- Apply device verification/authentication command flow.

Step 5.2 BLE Role and GATT

- Role: BLE Peripheral
- Service UUID: `0xFFFO`

Important characteristics:

- **Char2**: Primary app write path
- **Char4**: Notification output channel (also accepted as an alternate write path)

Step 5.3 Connection and Verification Flow

On app connection:

1. Device requests MTU and connection parameters.
2. After ~1 second, the device sends a welcome notification on Char4.
3. A timeout-based verification window opens.

Accepted verification command prefixes:

- `SOS_PUB_`
- `SOS_USER_`
- `SOS_TM_IP_`
- `SOS_TM_RV_`

Success behavior:

- BLE notification: EMR
- MQTT protocol extension:

<Tracking Protocol and Emergency Protocol>*<received_command>*\r\n

Failure behavior:

- Device Verification Failed.
- Connection terminating. Goodbye.

Step 5.4 BLE App Requirements

- Enable notifications on Char4 before sending commands.
- Use Char2 for writes (Char4 write is fallback).
- Handle verification timeout failures gracefully (no immediate reject payload for malformed commands).

Step 5.5 Communication and Messages at Each Event

Define event-wise BLE message behavior and resulting backend visibility.

- App sends command over BLE characteristic.
- Device validates and translates BLE command into internal parser command.
- Firmware generates operational packets/events.
- Resulting state and replies are propagated through MQTT topics.
- App receives BLE acknowledgment/notification as applicable.

Appendix A. Quick End-to-End Validation Flow

1. Confirm Step 1 whitelisting is complete.
2. Configure a test device per Step 2.
3. Install TLS certificate and verify secure MQTT connect (Step 3).
4. Validate telemetry publish and command/reply over MQTT (Step 4).
5. Validate BLE-triggered actions and corresponding MQTT/backend events (Step 5).

Appendix B. Implementation Notes

- Keep test and production credentials, certificates, and endpoints strictly isolated.
- Use structured logging for command source, command payload, and response mapping.
- Support queued/offline packet replay behavior in backend ingestion logic.